

SENTINEL

VERSION 1.0.0

AI-Powered Security & QA Testing — **Run it from your browser.**
Give it a URL. It does the rest.

Open `http://localhost:3000` → Enter a URL → Click "Launch Scan" → Done.

SENTINEL is a **web application** you run in your browser. Open the dashboard, enter a target URL, your product docs, and credentials. SENTINEL reads your documentation, logs into your app (even through enterprise SSO), explores every page, and then designs and runs security tests, functional checks, and edge-case exploration — all in a real Chromium browser. It learns from every run, so it gets smarter over time. No IDE required. No scripting. Baseline + AI attacks.

💡 Can also be run via **MCP** on IDEs like Cursor and VS Code for developer-integrated workflows.

0

SETUP
REQUIRED

16

ATTACK
CATEGORIES

5

LOGIN
METHODS
SUPPORTED

2

LAYERS:
BASELINE +
AI

4

REPORT
FORMATS
(HTML, MD,
JSON,
SARIF)

From One Prompt to a Full Security & QA Report

You give SENTINEL a URL, your product docs, and login credentials. It reads everything, logs in like a real user, explores the app, designs tests, runs them, and delivers a report — all automatically.

STEP 1

You Provide the Basics

Point SENTINEL at your app: the URL, your product documentation, login credentials, and sample data (e.g. a search term or a test user name). That's all you need.

STEP 2

SENTINEL Reads & Explores

It reads your docs to understand the app. Then it opens a real browser, logs in (even through enterprise SSO), and crawls every page — mapping forms, buttons, menus, and routes.

STEP 3

Baseline + AI Design Tests

First, SENTINEL generates **baseline attacks** from recon data — SQLi, XSS, CORS, IDOR, SSRF, traversal, and more on every input and API endpoint it found. Then the AI adds **app-specific attacks** tailored to your routes, tech stack, and business logic.

STEP 4

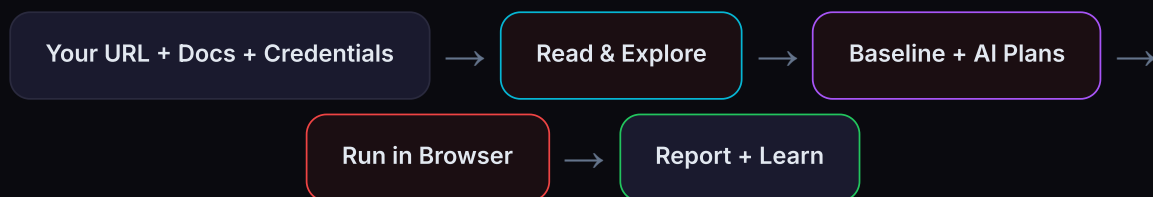
Tests Run in a Real Browser

Every test executes in a real Chromium browser — clicking buttons, filling forms, trying malicious inputs — just like a human tester would. Screenshots are captured as evidence.

STEP 5

Report & Remember

You get a clear report with every finding, severity, and screenshot proof. SENTINEL remembers what worked — the next scan starts smarter.



What Happens at Each Step

STEP 1

You Say What to Test

In plain English, you tell SENTINEL: *"Here's my app URL, here's the documentation, here are the login credentials, and here's some example data."* One prompt. No configuration files, no setup wizards, no scripting.

STEP 2

It Understands Your App

SENTINEL reads your product documentation to learn what the app does — its pages, features, user flows, and business rules. Then it opens a real browser, logs in (handling SSO, multi-factor, and consent screens), and maps every page, form, and button it finds.

STEP 3

Baseline + AI Test Plans

SENTINEL uses a **two-layer approach**. First, it generates **baseline attacks** directly from recon data — deterministic SQLi, XSS, CORS, IDOR, SSRF, traversal, auth bypass, CSRF, accessibility, and UX checks on every input and API endpoint discovered. Then the AI adds **app-specific attacks** tailored to your routes, tech stack, and business logic. Baseline ensures consistent coverage; AI adds depth.

STEP 4

Real Browser, Real Testing

Tests run in a real Chromium browser — not a proxy or a packet sniffer. SENTINEL clicks buttons, fills out forms, submits data, navigates between pages, and tries malicious inputs. Every action is captured with a screenshot so you can see exactly what happened.

STEP 5

Clear Reports, Growing Intelligence

Results come back as a prioritized list: critical issues at the top, passing checks at the bottom. Reports are available in HTML, Markdown, and formats that plug into your CI/CD pipeline. Every successful finding is saved — the next time you scan, SENTINEL already knows what worked and starts from a smarter baseline.

THE KEY DIFFERENCE

Two Layers: Baseline + AI

Traditional scanners ship with a static database of thousands of generic attack patterns and run them all blindly. SENTINEL takes a smarter approach: **baseline attacks** are generated deterministically from your app's actual recon data (every input, every API endpoint, every form), ensuring consistent coverage. Then the **AI layer** adds bespoke attacks tailored to your specific tech stack and business logic — finding issues that neither generic scanners nor static baselines would catch.

Brief Tech Flow

The 10-step pipeline from clicking "**Launch Scan**" in the web dashboard to a full report.



Step 6 is the key. In Rapid7/AppScan, steps 1–5 feed a static rule engine. In SENTINEL, recon data first generates **baseline attacks** — deterministic SQLi, XSS, CORS, IDOR, SSRF, traversal, and more on every discovered input and API endpoint. Then the GitHub Models API (GPT-4o) *reasons* about what else to test — adding app-specific attacks the baseline can't generate. The AI knows this app uses CSRF tokens, so it tests CSRF bypass. It read that `/folder/:folderId` is a route, so it tests IDOR. Baseline ensures consistency; AI adds intelligence.

Think of It This Way

Traditional scanners are like a locksmith who tries the same 10,000 keys on every door. Most don't fit. It's slow, noisy, and misses custom locks.

SENTINEL is like hiring a security team: one expert systematically checks every door and window (baseline), while another reads the building blueprints and picks specialized tools for the custom locks (AI). Together, they're faster, smarter, and more thorough — and they never miss the basics.

How SENTINEL Is Built — And Where AI Lives

SENTINEL is a **web application** powered by a Next.js dashboard and the GitHub Models API (GPT-4o). The AI model is the brain — it reads docs, analyzes recon data, generates every test, and decides what to attack. The server provides the hands: a real browser, authentication, crawling, and evidence capture. *It can also run as an MCP server inside IDEs like Cursor/VS Code.*

AI Layer The Brain

 LLM (Claude / GPT)

 Doc Understanding

 Test Plan Generation

 Payload Design

 Vulnerability Analysis

Web Dashboard The Interface

 Next.js Dashboard

 SSE Scan API

 Real-Time Progress

 Report Downloads

 Also: MCP Server (IDE)

Browser Engine The Hands

 Playwright + Chromium

 Auth (5 types)

 Screenshot Capture

 Form / Cookie / Storage Inspection

Data & Memory The Memory

 URL-Scoped Knowledge Base

 Effective Payloads

 Scan History

 App Profiles

Output The Reports

[HTML Report](#)[Markdown Report](#)[SARIF \(CI/CD\)](#)[Screenshot Evidence](#)

Where AI Is Leveraged

SENTINEL uses a two-layer approach: **baseline attacks** from recon data ensure consistent coverage, while the **AI layer** adds app-specific intelligence. Here's how each layer contributes:

BASELINE

Deterministic Attacks from Recon

The baseline engine generates attacks directly from crawl data: SQLi and XSS on every text input, CORS probes, IDOR on API endpoints, SSRF on preview/proxy URLs, path traversal, auth bypass, CSRF on forms, accessibility audits, and UX checks. These run every scan, ensuring consistent results.

AI DECIDES

Reads & Understands Documentation

The LLM reads your product docs (markdown, text) and extracts routes, features, business rules, tech stack, and user flows. This understanding shapes the app-specific tests it generates.

AI DECIDES

Generates App-Specific Attacks

Beyond the baseline, the LLM generates bespoke attacks tailored to your specific app — custom payloads for your tech stack, business logic tests, and edge cases that a deterministic engine can't anticipate.

AI DECIDES

Sets Success Indicators

The LLM decides what constitutes a "finding" for each AI-generated test. The baseline uses heuristic detection — pattern matching for SQL errors, XSS reflection, sensitive data exposure, and file content signatures.

AI DECIDES

Prioritizes & Triage Results

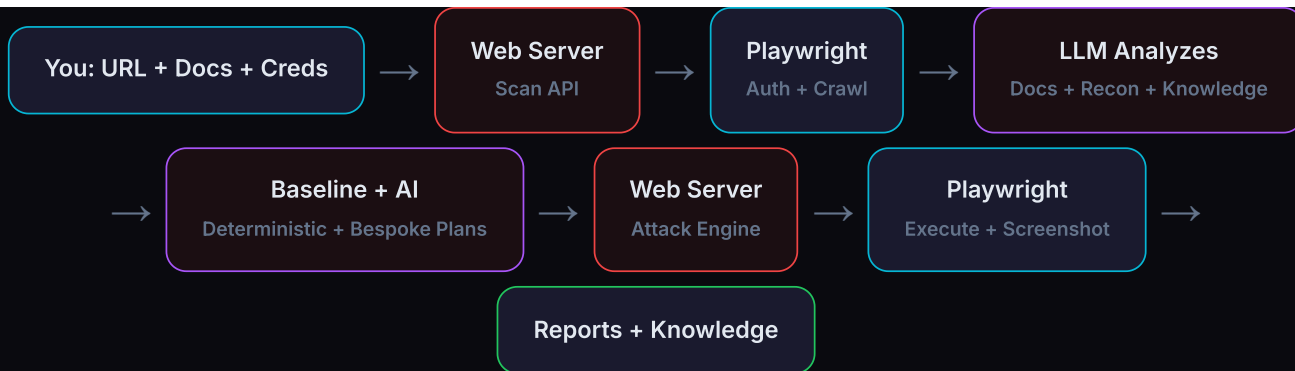
The LLM assigns severity (critical/high/medium/low) based on real-world impact, not a generic formula. It explains findings in context of your specific application.

TOOL EXECUTES

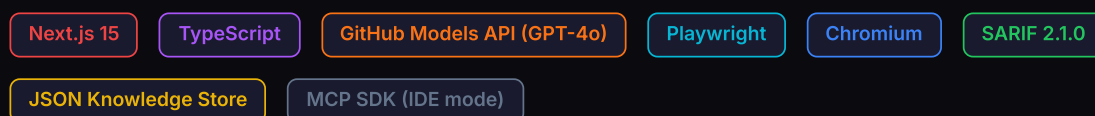
Browser Automation & Evidence

Playwright drives a real Chromium browser: navigating pages, filling forms, clicking buttons, capturing screenshots. Both baseline and AI attacks execute in the same real browser — faithfully and with evidence.

Data Flow: One Scan, End to End



Tech Stack



SENTINEL Remembers What Works

Every time SENTINEL tests an app, it saves what it learned. The next scan starts from a smarter baseline — it knows your app's structure, what tests found issues before, and what passed. Each app has its own separate memory.



App Understanding

Pages, features, and business rules learned from your product docs



Test History

A record of every scan: what was tested, what passed, what failed



What Worked Before

Attacks and tests that found real issues are saved and reused in future scans



Reusable Test Plans

Test plans built from your docs can be stored and re-run any time

HOW IT WORKS

One Prompt Is All You Need

You provide the URL, your product documentation, login credentials, and some example data. SENTINEL reads the docs to understand the app, logs in, explores every page, and brings all that context to the AI for test planning.

PER-APP MEMORY

Each App Has Its Own Brain

Testing App A doesn't mix with App B. Each application gets its own isolated memory — its own history, its own effective tests, its own learned patterns. You can test dozens of apps and each one has a clean, separate context.

CONTINUOUS IMPROVEMENT

Smarter With Every Scan

Found a vulnerability on Monday? SENTINEL remembers the exact test that found it. On Tuesday, it starts with that knowledge and builds on it. Close the tool, reopen it next week — the memory persists.

Inside `sentinel-knowledge.json`

This local JSON file is SENTINEL's long-term memory. Every scan writes to it. Every future scan reads from it. Think of it as the AI's personal pentester notebook.

scanHistory[] Audit Trail	Complete record of every scan — timestamp, pages scanned, attacks executed, findings summary. Currently 11 scans across 4 targets.
effectivePayloads[] What Works	Every payload that successfully found a vulnerability, tagged by category, target type, and app. Currently 68 payloads saved. Fed back to the AI on the next scan so it reuses proven attacks.
appId Scoping Isolation	Payloads are scoped per target URL. Knowledge for <code>app-a.example.com</code> never bleeds into <code>app-b.example.com</code> . Each app gets its own clean context.

11

SCANS RECORDED

68

EFFECTIVE PAYLOADS

4

TARGETS TRACKED

10

ATTACK CATEGORIES

SCAN 1

Cold Start

No prior knowledge. AI generates attacks purely from documentation + recon data.

SCAN 5

Warm

40+ effective payloads saved. AI reuses
Origin: <https://evil.com> (known to work)

Discovers initial vulnerabilities like CORS reflection and storage issues.

and focuses new effort on unexplored attack surfaces.

SCAN 11

Current

68 effective payloads. AI knows exactly which attacks work against each target. Scans are faster, more precise, fewer wasted attempts. Knowledge compounds.

Effective Payloads (sample)

cors: Origin: https://evil.com

csrf: no-csrf-header → 403

storage: CSRFToken in localStorage

storage: Username in localStorage

auth: /documents unauth → login

traversal: valid-id%00.html

exploratory: /nonexistent-route → error

functional: /home → hy-shell-header

Six Types of Testing, One Tool

Most tools do one thing. SENTINEL replaces your security scanner, your QA tester, your input validation suite, your accessibility auditor, your UX checker, and your exploratory tester — all in a single run.

SECURITY TESTING

Can a Hacker Break In?

SENTINEL tries to hack your app the way a real attacker would: SQL injection, cross-site scripting, bypassing login, accessing files it shouldn't, and more. Every attempt is documented with screenshot evidence.

FUNCTIONAL TESTING

Does the App Actually Work?

Does search return the right results? Does login work with valid credentials? Do dialogs open and display the correct fields? SENTINEL tests your app's features like a QA engineer would.

INPUT VALIDATION

What Happens With Bad Input?

Special characters, extremely long text, empty fields, foreign language characters. SENTINEL tests every input field to make sure your app handles unexpected data gracefully instead of crashing or leaking information.

EXPLORATORY TESTING

What Did Nobody Think to Test?

SENTINEL explores like a curious user: trying invalid URLs, clicking things in unexpected order, entering boundary values. It finds the edge cases that scripted tests miss because nobody thought to write a test for them.

ENTERPRISE LOGIN SUPPORT

Works With Your SSO

SENTINEL handles enterprise single sign-on automatically: it clicks the login button, gets redirected to your identity provider, fills in credentials, handles consent screens, and waits for the redirect back. Works with Hyland, Okta, Azure AD, and more.

API SECURITY

Tests Your APIs Too

SENTINEL doesn't just test web pages — it probes your JSON APIs for SSRF, IDOR, mass assignment, information disclosure, and insecure direct object references. It reads your API docs and generates targeted API-specific attacks.

PIPELINE-READY REPORTS

Results You Can Act On

Get results as a visual HTML report, a Markdown summary, or in SARIF format that plugs directly into GitHub's Security tab. Every finding includes CWE references and remediation recommendations.

ACCESSIBILITY

WCAG Compliance Checks

SENTINEL audits your app for accessibility violations: missing alt text, unlabeled form inputs, broken heading hierarchy, missing skip-to-content links, and missing lang attributes. Catches issues that block screen readers and keyboard users.

UX TESTING

Usability & Quality Checks

Broken images, dead links, slow page loads, unlabeled inputs, and content overflow. SENTINEL checks the user experience basics that manual QA often misses, ensuring your app works well for real users.

WEB DASHBOARD

Beautiful Real-Time UI

Run SENTINEL from your browser — enter a URL, watch the scan in real-time with live progress, and get an interactive results dashboard with severity cards, risk gauge, category breakdown, and downloadable reports. This is the primary way to use SENTINEL.

Authentication Types

form-login

Auto-fills username/password on a login form and submits

idp-login

Clicks app login button, fills IDP credentials, handles OAuth consent, waits for redirect

cookie

Sets session cookies directly in the browser context

bearer

Adds Authorization: Bearer header to all requests

basic

HTTP Basic Authentication with username/password

16 Attack Categories

SQL Injection

CRITICAL

DB-specific payloads tailored to the tech stack

Cross-Site Scripting

HIGH

Reflected and stored XSS with DOM verification

Auth Bypass

CRITICAL

Unauthenticated access + credential brute-force

Path Traversal

CRITICAL

SSRF

CRITICAL

IDOR

HIGH

Directory traversal via URL
params and file inputs

Server-side request forgery via
URL proxy endpoints

Insecure direct object references
on API endpoints

Info Disclosure HIGH

Debug endpoints, leaked
credentials, verbose errors

Open Redirect MEDIUM

Unvalidated redirects to external
domains

CORS / CSRF HIGH

Wildcard origins, missing CSRF
tokens, cookie issues

Storage Exposure MEDIUM

Tokens, API keys, credentials in
browser storage

Input Validation MEDIUM

Null bytes, CRLF, unicode, long
strings, special chars

Functional Tests QA

Navigation, dialogs, create
operations, search, logout

Exploratory Tests EXPLORE

Edge cases, invalid IDs, deep
routes, boundary values

DoS / ReDoS MEDIUM

Resource exhaustion,
catastrophic regex backtracking

Accessibility (WCAG)

A11Y

Missing alt text, labels, heading
hierarchy, skip links, contrast

UX / Usability ux

Broken images, dead links, slow
loads, unlabeled inputs

Run SENTINEL From Your Browser

SENTINEL v1.0.0 runs as a standalone Next.js web application. Open the dashboard, enter a URL, watch the scan in real-time, and get an interactive results page with downloadable reports. Powered by GitHub Models API (GPT-4o). No IDE needed.

REAL-TIME

Live Scan Progress

Watch every phase of the scan as it happens: reconnaissance, AI planning, attack execution, and findings appear in real-time via Server-Sent Events. No page refreshes, no waiting.

AI-POWERED

GitHub Models API (GPT-4o)

SENTINEL uses GitHub's Models API (GPT-4o) to generate attack plans. Set your GITHUB_TOKEN environment variable and the AI handles everything — no configuration, no API keys to manage.

INTERACTIVE

Rich Results Dashboard

Severity cards with animated counters, risk gauge, expandable finding cards with evidence and remediation, category breakdown chart, and one-click report downloads in HTML, Markdown, SARIF, and JSON.

Dashboard Features

✓ URL input with auth config

✓ Real-time SSE progress stream

✓ Animated severity counters

✓ Risk score gauge (0–100)

✓ Expandable finding cards

✓ Category breakdown chart

✓ Evidence + reproduction steps

✓ CWE + remediation per finding

✓ HTML/MD/SARIF/JSON downloads

✓ Dark theme, responsive design

✓ GitHub Models API (GPT-4o)

✓ 16 attack categories

```
# Quick start
$ cd web && npm install
$ echo "GITHUB_TOKEN=ghp_your_token" > .env.local
$ npm run dev
→ Dashboard ready at http://localhost:3000
```

VulnShop — A Built-In Demo App

SENTINEL ships with VulnShop, a deliberately vulnerable demo app so you can see it work immediately. One command launches the app, one prompt runs the full scan.

5+

PAGES SCANNED

30+

TESTS EXECUTED

20+

ISSUES FOUND

16

ATTACK
CATEGORIES

VulnShop Findings (Security + API + QA)

- CRITICAL** SQL Injection on search — `' OR 1=1 --`
- CRITICAL** SQL Injection on API — `/api/products?category=' UNION SELECT`
- CRITICAL** Auth bypass on `/admin` — no authentication required
- CRITICAL** Path traversal — `../../package.json`
- CRITICAL** SSRF via `/api/preview` — server fetches internal URLs
- HIGH** IDOR — `/api/users` exposes passwords and API keys
- HIGH** IDOR — `/api/orders/:id` accessible without auth
- HIGH** Info leak — `/api/debug` exposes server environment
- HIGH** Reflected XSS on search — `<script>alert(1)</script>`
- HIGH** Stored XSS on feedback form
- HIGH** Wildcard CORS — `Access-Control-Allow-Origin: *`
- HIGH** Missing CSRF token on feedback form
- MEDIUM** Open redirect via `/redirect` endpoint

MEDIUM Session cookie without Secure flag

MEDIUM Sensitive data in localStorage (JWT, API keys, credentials)

MEDIUM Header injection via /api/export filename

FUNCTIONAL Search returns correct results for valid queries

FUNCTIONAL Login redirects to admin on success

EXPLORE 404 handler leaks all available routes

A11Y Images missing alt text (WCAG 1.1.1)

A11Y Form inputs missing associated labels

A11Y No skip-to-content navigation link

UX Broken images (404) on product page

UX Dead links with empty href attributes

```
# 1. Start the demo app
$ cd demo-app && node server.js
→ VulnShop at http://localhost:3001

# 2. Start the web dashboard
$ cd web && npx next dev
→ Dashboard at http://localhost:3000

# 3. Open browser, enter URL, click "Launch Scan"
# Real-time results stream to the dashboard
→ 24 issues found across 16 categories
```

🔧 Alternatively, SENTINEL can also be run via **MCP** inside Cursor or VS Code for IDE-integrated workflows.

SENTINEL vs. Traditional DAST

Tools like **Rapid7 InsightAppSec**, **HCL AppScan**, Burp Suite, and OWASP ZAP are established DAST scanners. They work by firing thousands of pre-built payloads at every input and matching responses against known vulnerability signatures. SENTINEL takes a fundamentally different approach.

The Core Difference: AI Reasoning vs. Signature Matching

Rapid7 / AppScan / Burp / ZAP ship with a database of thousands of known attack signatures. They crawl your app, fire every payload at every input, and check if the response matches a known vulnerability fingerprint. They don't *understand* your app — they just pattern-match.

SENTINEL uses a two-layer approach. **Baseline attacks** are generated deterministically from your app's actual recon data — every input, every API endpoint, every form gets tested with targeted payloads. Then the **AI layer** (GPT-4o) reads your documentation, studies the recon, and *designs bespoke attack plans* for your specific business logic. Baseline ensures consistency; AI adds intelligence.

CAPABILITY	SENTINEL	RAPID7 INSIGHTAPPSEC	HCL APPSCAN	BURP SUITE / ZAP
Attack generation	Baseline from recon + AI per-app	Pre-built signature DB	Pre-built signature DB	Pre-built signature DB
Attack categories	16 (incl. SSRF, IDOR, a11y, UX)	OWASP Top 10	OWASP Top 10	OWASP Top 10
Reads product docs	docPath param	No	No	No
Business logic testing	AI-driven (IDOR, privilege esc.)	No	No	No
API security testing	SSRF, IDOR, info-leak, mass assignment	Limited	Limited	Manual

CAPABILITY	SENTINEL	RAPID7 INSIGHTAPPSEC	HCL APPSCAN	BURP SUITE / ZAP
Remediation guidance	Per-finding with CWE references	Generic	Generic	Manual
Learning across scans	Knowledge base persists	Each scan independent	Each scan independent	Session only
Browser storage audit	localStorage, sessionStorage, IndexedDB	Cookies only	Cookies only	Cookies only
IDP / OAuth / OIDC auth	Native (idp-login)	Manual macro recording	Manual macro recording	Manual macro recording
Setup time	Minutes (one call)	Hours — scan policies, auth macros, scope rules	Hours — scan config, exclusions	Moderate — proxy config
False positive rate	Low (AI validates context)	High (signature matching)	High (signature matching)	Moderate to high
Functional + QA testing	Built-in (same run)	Security only	Security only	Security only
Exploratory testing	Built-in	No	No	No
Accessibility (WCAG)	Built-in (a11y category)	No	No	No
UX / Usability testing	Built-in (ux category)	No	No	No
Web dashboard	Standalone Next.js UI (primary)	Web console	Standalone app	Plugin
Payloads per scan	30–60 (baseline + AI targeted)	10,000+ (brute-force)	10,000+ (brute- force)	Thousands
Scan duration	~5 minutes	2–8 hours typical	1–6 hours typical	30 min–hours
Real browser execution	Chromium (Playwright)	Headless engine	Headless engine	Proxy-based

CAPABILITY	SENTINEL	RAPID7 INSIGHTAPPSEC	HCL APPSCAN	BURP SUITE / ZAP
Screenshot evidence	Automatic per finding	No	Limited	Manual
Example data input	exampleData param	No	No	No
Primary interface	Web dashboard (+ MCP for IDEs)	Web console only	Standalone app	Plugin
SARIF / CI output	Built-in	Yes	Yes	Plugin
Cost	Free / MIT + LLM API costs	\$10K–\$100K+/yr	\$10K–\$80K+/yr	Free (ZAP) / \$449+/yr (Burp)

Rapid7 / AppScan Workflow

1. Configure scan policy (hours)
2. Record auth macro manually
3. Define scope and exclusions
4. Fire 10,000+ generic payloads
5. Wait 2–8 hours for scan
6. Triage 200+ findings (many false positives)
7. Repeat from scratch next time

SENTINEL v1.0.0 Workflow

1. Open dashboard, enter URL + docs + credentials
2. AI authenticates (even IDP/OAuth)
3. AI reads docs, understands the app
4. Baseline + AI design 30+ precision attacks
5. Executes in ~5 minutes
6. 9 validated findings, low false positives
7. Knowledge base carries forward

KEY ADVANTAGE

Baseline + AI, Not Blind Signatures

Rapid7 and AppScan ship with a static database of thousands of known attack patterns and run them all blindly. SENTINEL generates baseline attacks from your actual recon data (every input, every API endpoint), then the AI adds app-specific tests. It tests what matters, consistently and intelligently.

KEY ADVANTAGE

Gets Smarter Over Time

Each scan builds on the last. Successful tests are remembered. Knowledge persists across sessions. Rapid7 and AppScan start from zero every time — SENTINEL picks up right where it left off.

KEY ADVANTAGE

Replaces Four Tools With One

No DAST tool — not Rapid7, not AppScan, not Burp — does security testing, functional QA, input validation, and exploratory testing in one run. SENTINEL covers all four, reducing tool sprawl and giving you a single source of truth.

Open Your Browser. Launch a Scan. Get a Full Report.

SENTINEL v1.0.0 — Run it from the web. Give it a URL, your docs, and credentials. It does the rest.

```
cd web && npx next dev → http://localhost:3000
```

Also available via MCP on Cursor / VS Code

Open Source (MIT) • v1.0.0 • 16 Attack Categories • Web Dashboard • Real Browser Testing • Baseline + AI Attacks • Per-App Memory • Accessibility & UX Testing • API Security • Remediation Guidance